

JavaScript Foundations

Comprehensive Study Guide & Core Syntax Reference | SHS ICT Curriculum

CLIENT-SIDE SCRIPTING

ECMAScript 6+

DOM MANIPULATION

ALGORITHM LOGIC

⚡ 1. INTRODUCTION TO JAVASCRIPT & STATEMENTS

JavaScript is a high-level, interpreted programming language that enables interactive web pages. While HTML structures content and CSS styles layout, JavaScript provides behavioral logic and dynamic functionality.

What is a JS Statement?

A JavaScript statement is a step-by-step instruction executed by the browser engine to perform an action.

- Composed of keywords, values, operators, and expressions.
- Typically ends with a semicolon (`;`) for clean statement separation.
- Executed sequentially from top to bottom.

Core Output Statements

Ways to output or interact with users in JavaScript:

```
// DOM Output (Modifying web content)
document.getElementById("demo").innerHTML = "Hello!";

// Browser Console Log (Debugging)
console.log("Execution status: OK");

// Popup Alert Box
alert("Form submitted successfully!");
```

📦 2. VARIABLES, SCOPE & DATA TYPES

Variables act as labeled storage containers in computer memory for holding data values.

Keyword	Scope	Reassignable?	Redeclarable?	Best Practice Usage
<code>const</code>	Block Scope <code>{ }</code>	❌ No	❌ No	Default choice for values/references that shouldn't change.
<code>let</code>	Block Scope <code>{ }</code>	✅ Yes	❌ No	Use for counters, loops, or variables updated later.
<code>var</code>	Function Scope	✅ Yes	✅ Yes	Legacy ES5 syntax (avoid in modern JS due to hoisting nuances).

Data Types Summary

Category	Type	Description	Example Syntax
Primitive	String	Textual data enclosed in single/double quotes or backticks	<code>const name = "Marionne";</code>
	Number	Integers and decimal numbers for math operations	<code>let score = 95.5;</code>
	Boolean	Logical flags representing <code>true</code> or <code>false</code>	<code>let isPassed = true;</code>
	Undefined / Null	Uninitialized variable vs intentional empty value	<code>let x; // undefined let y = null;</code>
Non-Primitive	Array	Ordered collection/list of indexed elements (0-indexed)	<code>const grades = [92, 95, 96, 98];</code>
	Object	Key-value pairs representing structured entities	<code>const student = { name: "John", grade: 12 };</code>

3. FUNCTIONS & EXECUTION LOGIC

Functions are reusable blocks of code designed to perform a specific task when invoked (called).

Function Syntax & Parameters

```
// Function Declaration
function calculateAverage(q1, q2, q3, q4) {
  let sum = q1 + q2 + q3 + q4;
  let avg = sum / 4;
  return avg; // Returns output
}

// Invoking / Calling Function
let finalGrade = calculateAverage(92, 95, 96, 98);
```

Arrow Functions (ES6)

Modern, concise syntax for writing function expressions:

```
// Arrow Function Syntax
const computeTotal = (price, tax) => price + (price * tax);

// Calling Arrow Function
let totalBill = computeTotal(550, 0.12);
```

4. CONTROL STRUCTURES & CONDITIONALS

Decision Making with `if...else` & Operators

```
const finalGrade = 95.25;

// Conditional Checks using Comparison and Logical Operators
if (finalGrade >= 90) {
  console.log("Academic Honors: Outstanding");
} else if (finalGrade >= 75) {
  console.log("Status: Passed");
} else {
  console.log("Status: Needs Improvement");
}
```

Comparison Operators

- `==` Equal to (Type Coercion)
- `===` Strict Equal to (Value & Type)
- `!=` / `!==` Not Equal / Strict Not Equal
- `>`, `<`, `>=`, `<=` Relational checks

Logical Operators

- `&&` (AND): Both conditions must be true
- `||` (OR): At least one condition is true
- `!` (NOT): Inverts boolean truthiness



5. PROBLEM SOLVING: ALGORITHM TO JAVASCRIPT IMPLEMENTATION

Sample Problem: Calculate Total Store Purchase

Scenario: Maria bought Rice (P550), Eggs (P210), and Bread (P95). Calculate total bill and output customer name with total amount to a webpage element.

Flowchart Shape Mapping

- **Oval (Terminator):** START / END
- **Rectangle (Process):** Variable assignment / Math (`total = priceRice + priceEggs + priceBread`)
- **Parallelogram (I/O):** Input values / Display output to user
- **Diamond (Decision):** Conditional check (e.g. `total > 1000?`)

Structured Pseudocode

```
START
  INITIALIZE customerName = "Maria"
  INITIALIZE priceRice = 550, priceEggs = 210, priceBread = 95
  COMPUTE totalAmount = priceRice + priceEggs + priceBread
  DISPLAY customerName AND totalAmount
END
```

Complete HTML & DOM JavaScript Solution

```
<!-- HTML Markup -->
<div id="billDisplay"></div>

<script>
  // JavaScript Implementation
  const customerName = "Maria";
  const priceRice = 550;
  const priceEggs = 210;
  const priceBread = 95;

  // Process calculation
  const totalAmount = priceRice + priceEggs + priceBread;

  // DOM Manipulation Statement
  document.getElementById("billDisplay").innerHTML =
    `<h3>Customer: ${customerName}</h3>` +
    `<p>Total Bill: P${totalAmount.toFixed(2)}</p>`;
</script>
```

 **Key Reviewer Tip:** Always verify element IDs in HTML match the exact string passed into `document.getElementById("elementID")`, including letter case!